

Отладка с использованием KGDB

KGDB — это отладчик ядра Linux, работающий совместно с GDB на хост-машине, позволяя отлаживать ядро на целевой машине через последовательный порт или сеть.

Сборка ядра с включёнными опциями для отладчика KGDB.

Основные опции ядра для отладки с KGDB

Для успешной отладки ядра Linux с помощью KGDB в конфигурации ядра должны быть включены следующие ключевые опции:

Опция конфигурации	Описание	Рекомендуемое значение
CONFIG_KGDB	Включение поддержки отладчика KGDB для ядра Linux	у (включено)
CONFIG_KGDB_SERIAL_CONSOLE	Использование последовательного порта для работы KGDB	у (включено)
CONFIG_DEBUG_INFO	Включение генерации отладочной информации DWARF	у (включено)
CONFIG_FRAME_POINTER	Сохранение указателей кадров для удобной трассировки стека	у (включено)
CONFIG_MAGIC_SYSRQ	Включение поддержки Magic SysRq клавиш для управления ядром и отладки	у (включено)
CONFIG_MAGIC_SYSRQ_DEFAULT_ENABLE	Значение по умолчанию для /proc/sys/kernel/sysrq, задаёт разрешённые SysRq команды	0x1b6 (обычное значение)
CONFIG_MAGIC_SYSRQ_SERIAL	Позволяет использовать Magic SysRq через последовательный порт (serial console)	у (включено)
CONFIG_DEBUG_INFO_REDUCED	Сокращение объёма отладочной информации (лучше отключить для полной отладки)	н (отключено)
CONFIG_RANDOMIZE_BASE	Рандомизация адресного пространства ядра (KASLR)	н (рекомендуется отключить)
CONFIG_KALLSYMS	Включение поддержки символов ядра, полезно для отладки	у (включено)
CONFIG_KALLSYMS_ALL	Включение всех символов в kallsyms	у (включено)

Примечания

- Для успешной генерации отладочной информации в ядре нужно обязательно отключить опцию CONFIG_DEBUG_INFO_NONE, иначе CONFIG_DEBUG_INFO не будет собираться.
- Включение CONFIG_MAGIC_SYSRQ критично для возможности отладки загруженных модулей ядра. Без этой опции отладка модулей в уже работающей системе становится

крайне затруднительной.

3. Опции `CONFIG_MAGIC_SYSRQ_DEFAULT_ENABLE` и `CONFIG_MAGIC_SYSRQ_SERIAL` позволяют настроить, какие комбинации SysRq будут доступны и поддерживают их работу через последовательный порт, что удобно для удалённой отладки.
4. Отключение `CONFIG_RANDOMIZE_BASE` (KASLR) упрощает работу с отладкой, поскольку адреса ядра и модулей не будут динамически изменяться при каждой загрузке. Отключить KASLR можно добавлением параметра загрузки ядра `nokaslr` в строку `GRUB_CMDLINE_LINUX_DEFAULT` в файле `/etc/default/grub` и последующим обновлением конфигурации загрузчика командой `sudo update-grub`.

Сценарии отладки с KGDB

Отладка с помощью **KGDB** может применяться как для ранних этапов загрузки ядра, так и для уже загруженных модулей. Основные моменты:

Отладка ядра на ранних этапах загрузки

1. Предназначена для выявления проблем, возникающих ещё до загрузки модулей и запуска системных сервисов.
2. Ядро запускается с параметрами загрузчика, например `kgdbwait kgdboc=ttyS0,115200`, которые заставляют его остановиться и ждать подключения отладчика.
3. Позволяет поставить breakpoints на ключевые функции инициализации, например, `start_kernel`, `rest_init`, чтобы пошагово отслеживать процесс загрузки.
4. Важна для диагностики проблем, когда система даже не загружается полностью.

Отладка конкретных модулей ядра в работающей системе

1. Позволяет отлаживать код модулей, которые уже загружены и работают в рамках действующего ядра.
2. Требуется подключение к запущенной системе, загрузка символов модуля в GDB и установка breakpoint'ов на функции модуля.
3. Используется для отладки драйверов устройств, файловых систем и других компонентов, реализованных в виде модулей.
4. Зависит от включения опции `CONFIG_MAGIC_SYSRQ` для возможности безопасного прерывания выполнения ядра в любой момент и входа в режим отладки.
5. Подходит для динамического анализа поведения модулей без необходимости перезагружать систему.

Итог:

Это позволяет охватить основные сценарии, когда нужно:

- Либо контролировать загрузку ядра с самого начала,
- Либо исследовать работу отдельных частей ядра уже во время выполнения.

Такое разделение помогает понимать, что и когда можно отлаживать с помощью KGDB и выбрать подходящую стратегию отладки для текущей задачи.

Подготовка виртуальной машины с использованием virt-manager

Для отладки ядра Linux с помощью **KGDB** в виртуальной машине через **virt-manager** необходимо подготовить VM с поддержкой последовательного порта.

Заходим в настройки VM (нажимаем на виртуальную машину дважды, затем Вид->Подробности).

Добавляем последовательный порт (Добавить оборудование->Последовательное(Тип: `Unix` , путь `/tmp/kgdb-debug`))

Пошаговая отладка ядра на ранних этапах загрузки с KGDB.

1. Подготовка ядра(выше в лекции).
2. Подготовка VM(выше в лекции).
3. Настройка параметров загрузки ядра. В `/etc/default/grub` добавьте к параметрам загрузки ядра следующие опции:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash kgdboc=ttyS0,115200 kgdbwait"
```

Примечание:

`kgdboc=ttyS0,115200` — указывает, что отладчик **KGDB** будет использовать последовательный порт `ttyS0` с скоростью `115200` бод.

`kgdbwait` — заставляет ядро остановиться на раннем этапе загрузки и ждать подключения отладчика.

Обязательно обновите конфигурацию загрузчика GRUB, выполнив команду

```
sudo update-grub
```

4. Запустите виртуальную машину с подготовленным ядром и параметрами загрузки. Ядро загрузится и остановится в ожидании подключения GDB.
5. Подключение **GDB** к целевой системе. На хост-машине запустите **GDB** с образом ядра с символами, вот так:

```
gdb /path/to/vmlinux
```

Подключитесь к целевой системе через файл `/tmp/kgdb-debug` :

```
(gdb) target remote /tmp/kgdb-debug
```

6. Установка точек останова. Установите breakpoints на функции инициализации ядра, например:

```
(gdb) break start_kernel
```

Можно также установить breakpoints на другие важные функции ранней загрузки.

7. Продолжение и пошаговая отладка. Запустите выполнение ядра командой:

```
(gdb) continue
```

Отладчик **KGDB** остановится на установленных точках останова.

Используйте команды **gdb** для просмотра регистров, памяти, переменных, пошагового выполнения (`stepi`, `nexti`), изменения значений и др.

Отладка конкретных модулей ядра в работающей системе

1. Подготовка ядра(выше в лекции).
2. Подготовка вм(выше в лекции).
3. Сборка модуля с опциями для отладочных символов. Добавьте в Makefile вашего модуля

```
ccflags-y := -O1 -g
```

Примечание:

`-O1` — это уровень оптимизации компилятора, который обеспечивает базовые оптимизации без сильного искажения кода.

`-g` — включает генерацию отладочной информации (символы, информация о переменных и функциях).

4. Скиньте на хостовую систему `.ko` файл с отладочными символами.

5. Загрузка модуля в `vm`. В `vm` загрузите модуль который хотите отладить:

```
sudo insmod test_kgdb_module.ko
```

6. Определение адресов секций загруженного модуля. Узнайте физические адреса секций `.text`, `.data` и `.bss` модуля с помощью:

```
cat /sys/module/test_kgdb_module/sections/.text
cat /sys/module/test_kgdb_module/sections/.data
cat /sys/module/test_kgdb_module/sections/.bss
```

7. Подключение **GDB** к отладочным символам из `vmlinux`. На хост-машине запустите **GDB** с образом ядра с символами, вот так:

```
gdb /path/to/vmlinux
```

8. Загрузка символов модуля в **GDB**. На хосте загрузите символы модуля и укажите адреса секций:

```
(gdb) add-symbol-file /path/to/test_kgdb_module.ko 0x<text_addr> -s .data 0x<data_addr> -s .b
```

Где `0x<text_addr>`, `0x<data_addr>`, `0x<bss_addr>` — адреса, полученные на предыдущем шаге.

Почему просто `-g` не достаточно?

Опция `-g` даёт отладочную информацию с относительными адресами, но не указывает, куда именно модуль был загружен в физическую память ядра.

Таким образом, **GDB** сам по себе не знает расположение секций модуля в памяти ядра цели, а для эффективной отладки требуется синхронизировать адреса.

9. Активация и регистрация драйвера ввода-вывода **KGDB** (`kgdboc`). Выполните команду:

```
echo "ttyS0,115200" > /sys/module/kgdboc/parameters/kgdboc
```

В `dmesg` должно появиться сообщение:

```
kgdb: Registered I/O driver kgdboc.
```

Пояснение:

Команда выше используется для настройки и регистрации драйвера ввода-вывода **KGDB** с конкретным устройством, например, последовательным портом `ttyS0` на скорости `115200` бод. Это позволяет ядру использовать последовательный порт для связи с отладчиком **KGDB**.

10. Вызов отладчика KGDB с помощью Magic SysRq.

```
echo g > /proc/sysrq-trigger
```

Эта команда используется для принудительной остановки выполнения ядра и входа в отладчик **KGDB** с помощью механизма Magic SysRq.

11. Подключитесь к целевой системе через файл /tmp/kgdb-debug :

```
(gdb) target remote /tmp/kgdb-debug
```

12. Установка точек останова в модуле. Установите breakpoints в функциях модуля, например:

```
(gdb) break my_module_function
```

13. Продолжение и пошаговая отладка. Запустите выполнение ядра командой:

```
(gdb) continue
```

Отладчик **KGDB** остановится на установленных точках останова.

Используйте команды **gdb** для просмотра регистров, памяти, переменных, пошагового выполнения (`stepi`, `nexti`), изменения значений и др.

Ресурсы для самостоятельного изучения:

- [How to use KGDB](#)
- [Using kgdb, kdb and the kernel debugger internals](#)
- [Погружение в недра. Разбираем kernel exploitation](#)
- [Linux Magic System Request Key Hacks](#)

Отладка с использованием KASAN (Kernel Address Sanitizer)

KASAN — это динамический инструмент для обнаружения ошибок доступа к памяти в ядре Linux. Он помогает выявлять типичные ошибки, такие как выход за пределы выделенного буфера, использование уже освобождённой памяти (use-after-free), двойное освобождение и другие проблемы, приводящие к сбоям и уязвимостям.

Основные возможности KASAN

- Проверка всех операций с памятью во время выполнения ядра.
- Автоматическое обнаружение ошибок с подробным описанием и трассировкой стека.
- Повышение безопасности и стабильности ядра.
- Поддержка большинства архитектур процессоров.

Включение и настройка KASAN

Для успешной отладки модулей ядра Linux с помощью KASAN в конфигурации ядра должны быть включены следующие ключевые опции:

Опция	Описание	Значение
CONFIG_KASAN	Включить Kernel Address Sanitizer	y
CONFIG_KASAN_INLINE	Включить инлайн проверки для повышения производительности	y
CONFIG_DEBUG_INFO	Включить генерацию отладочной информации DWARF	y
CONFIG_KASAN_STACK	Включить отслеживание состояния стека (рекомендуется)	y

Параметры загрузки ядра с KASAN

1. В файл конфигурации загрузчика (`/etc/default/grub`) добавьте параметр:

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash kasan=1"
```

Этот параметр явно включает **KASAN** при загрузке ядра. Однако, если ядро собрано с включённой опцией `CONFIG_KASAN` , **KASAN** будет включён по умолчанию без необходимости указывать `kasan=1` в командной строке загрузки.

Когда этот параметр полезен ?

- Если установлен параметр `kasan=0`, то KASAN будет отключён, несмотря на то, что он собран в ядре, что полезно для отключения KASAN в продуктивных системах, где важна производительность.

Можно использовать параметр `kasan=early` для активации **KASAN** на самых ранних этапах загрузки ядра, что позволяет выявлять ошибки сразу при инициализации.

2. Обновление загрузчика

После внесения изменений в файл `/etc/default/grub` выполните:

```
sudo update-grub
```

для обновления конфигурации загрузчика, а затем перезагрузите систему.

Как использовать KASAN (пример).

Возьмем вот такой код:

```
#include <linux/fs.h>
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/proc_fs.h>
#include <linux/slab.h>
#include <linux/uaccess.h>

static struct proc_dir_entry *entry;

static u8 *test_kasan_buffer;

static ssize_t test_kasan_write(struct file *filep, const char __user *buf, size_t length, lo
{
    unsigned long n;
    n = copy_from_user(test_kasan_buffer, buf, length);
    return length;
}

static const struct proc_ops test_kasan_ops = {
    .proc_write = test_kasan_write,
};

static int __init test_kasan_init(void)
{
    test_kasan_buffer = kmalloc(14, GFP_KERNEL);
    if (!test_kasan_buffer)
        return -ENOMEM;

    entry = proc_create("test_kasan", S_IWUSR, NULL, &test_kasan_ops);
    if (!entry)
        return -ENOMEM;

    return 0;
}

static void __exit test_kasan_exit(void)
{
    kfree(test_kasan_buffer);
    remove_proc_entry("test_kasan", NULL);
}

MODULE_AUTHOR("KGD");
MODULE_DESCRIPTION("Testing KASAN");
MODULE_LICENSE("GPL");

module_init(test_kasan_init);
```



```
module_exit(test_kasan_exit);
```

Обратите внимание, что в реализации write нашего устройства `proc test_kasan_write` не проверяется длина записываемых данных перед попыткой записи в наш локальный буфер `test_kasan_buffer`, с помощью `copy_from_user`. Наш локальный буфер указывает на 14 байт выделенной памяти, поэтому любая запись в `/proc/test_kasan`-устройство длиной более 14 байт должна вызывать ошибку.

Вызываем ошибку

После компиляции модуля и создания виртуальной машины, которая загружает ядро с параметром `CONFIG_KASAN`, мы сможем вызвать ошибку KASAN после загрузки модуля с помощью чего-то вроде следующего:

```
echo "Hello, KASAN error" > /proc/test_kasan
```

Это должно вызвать ошибку и отобразить отчёт.

Анализируем отчет

Функции загрузки и сохранения также должны выводить отчёт для пользователя, если проверка памяти выявляет проблему с доступом к ней. Отчёт содержит информацию, полезную для отладки ошибки, а также трассировку стека, которая должна быть вам знакома, если вы уже сталкивались с ошибками ядра или паническими ситуациями. Ниже приведён простой пример:

```
=====
BUG: KASAN: slab-out-of-bounds in _copy_from_user+0x51/0x90
Write of size 19 at addr ffff888230c0d4a0 by task bash/879

CPU: 3 PID: 879 Comm: bash Tainted: G          O          5.4.0-rc5+ #16
Hardware name: QEMU Standard PC (Q35 + ICH9, 2009), BIOS ?
\20190711_202441-buildvm-armv7-10.arm.fedoraproject.org-2.fc31 04/01/2014
Call Trace:
 dump_stack+0x5b/0x90
 print_address_description.constprop.0+0x16/0x200
 ? _copy_from_user+0x51/0x90
 ? _copy_from_user+0x51/0x90
 __kasan_report.cold+0x1a/0x41
 ? _copy_from_user+0x51/0x90
 kasan_report+0xe/0x20
 check_memory_region+0x130/0x1a0
 _copy_from_user+0x51/0x90
 test_kasan_write+0x11/0x30 [test_kasan]
 proc_reg_write+0x110/0x160
 ? proc_reg_unlocked_ioctl+0x150/0x150
 ? __pmd_alloc+0x150/0x150
 ? __audit_syscall_entry+0x18e/0x1f0
 ? ktime_get_coarse_real_ts64+0x46/0x60
 ? security_file_permission+0x66/0x190
```

```
vfs_write+0xed/0x240
ksys_write+0xb4/0x150
? __ia32_sys_read+0x40/0x40
? up_read+0x10/0x70
? do_user_addr_fault+0x3da/0x560
do_syscall_64+0x5e/0x190
entry_SYSCALL_64_after_hwframe+0x44/0xa9
RIP: 0033:0x7fe39aa3c150
Code: 73 01 c3 48 8b 0d 20 6d 2d 00 f7 d8 64 89 01 48 83 c8 ff c3\
66 0f 1f 44 00 00 83 3d 4d ce 2d 00 00 75 10 b8 01 00 00 0f 05\
<48> 3d 01 f0 ff ff 73 31 c3 48 83 ec 08 e8 ee cb 01 00 48 89 04 24
RSP: 002b:00007fff5b7839d8 EFLAGS: 00000246 ORIG_RAX: 0000000000000001
RAX: ffffffffda RBX: 0000000000000011 RCX: 00007fe39aa3c150
RDX: 0000000000000011 RSI: 00007fe39b366000 RDI: 0000000000000001
RBP: 00007fe39b366000 R08: 000000000000000a R09: 00007fe39b35c740
R10: 0000000000000022 R11: 0000000000000246 R12: 00007fe39ad14400
R13: 0000000000000011 R14: 0000000000000001 R15: 0000000000000000
```

Allocated by task 894:

```
save_stack+0x1b/0x80
__kasan_kmalloc.constprop.0+0xc2/0xd0
0xffffffffc0008022
do_one_initcall+0x86/0x29f
do_init_module+0xf8/0x350
load_module+0x3e57/0x4120
__do_sys_finit_module+0x162/0x190
do_syscall_64+0x5e/0x190
entry_SYSCALL_64_after_hwframe+0x44/0xa9
```

Freed by task 614:

```
save_stack+0x1b/0x80
__kasan_slab_free+0x12c/0x170
kfree+0x90/0x240
xdr_free_bvec+0x1a/0x30
xpirt_release+0x10a/0x270
rpc_release_resources_task+0x14/0x70
__rpc_execute+0x253/0x590
rpc_async_schedule+0x44/0x70
process_one_work+0x476/0x760
worker_thread+0x73/0x680
kthread+0x18c/0x1e0
ret_from_fork+0x35/0x40
```

The buggy address belongs to the object at ffff888230c0d4a0

which belongs to the cache kmalloc-16 of size 16

The buggy address is located 0 bytes inside of

16-byte region [ffff888230c0d4a0, ffff888230c0d4b0)

The buggy address belongs to the page:

page:ffffea0008c30340 refcount:1 mapcount:0

mapping:ffff888236403b80 index:0xfffff888230c0d9c0

flags: 0x200000000000200(slab)

raw: 0200000000000200 fffffea0008b86700 0000001200000012 ffff888236403b80

raw: ffff888230c0d9c0 0000000080800079 00000001ffffffff 0000000000000000

```
page dumped because: kasan: bad access detected
```

Memory state around the buggy address:

```
ffff888230c0d380: 00 00 fc fc fb fb fc fc fb fb fc fc fb fb fc fc
ffff888230c0d400: fb fb fc fc fb fb fc fc fb fb fc fc fb fb fc fc
>ffff888230c0d480: fb fb fc fc 00 06 fc fc fb fb fc fc 00 00 fc fc
                        ^
ffff888230c0d500: 00 00 fc fc 00 00 fc fc 00 00 fc fc 00 00 fc fc
ffff888230c0d580: 00 00 fc fc 00 00 fc fc 00 00 fc fc 00 00 fc fc
=====
```

Обратите внимание, что в последнем разделе отчёта приведён шестнадцатеричный дамп соответствующей теневой памяти для доступа, вызвавшего ошибку. Интересующая нас строка в шестнадцатеричном дампе выделена `>`. Точный байт, кодирующий 8-байтовый раздел, в котором находился адрес, выделен `^`.

```
>ffff888230c0d480: fb fb fc fc 00 06 fc fc fb fb fc fc 00 00 fc fc
                        ^
```

Отчёт об ошибке будет содержать информацию:

- Тип сбоя (например, slab-out-of-bounds).
- Какое действие вызвало ошибку (например, запись "Write of size 19 at addr ...").
- Информацию о задаче (PID, имя процесса).
- Трассировку стека, показывающую последовательность вызовов функций до ошибки.
- Информацию об выделении и освобождении памяти.
- Детальный дамп теневой памяти с указанием проблемного байта.

Ресурсы для самостоятельного изучения KASAN

- [Kernel Address Sanitizer \(KASAN\) — официальная документация Linux kernel](#)
- [Статья на Хабре про KASAN: "KASAN — что это такое и зачем он нужен"](#)
- [Параметры ядра Linux — справочник на Microsin.net](#)
- [Введение в KASAN — LWN.net](#)
- [Google kernel sanitizers — официальный проект с описанием KASAN](#)
- [Интерактивное обучение Linux Kernel Labs: Debugging](#)

Отладка с использованием kmemleak (Kernel Memory Leak Detector)

kmemleak — это инструмент в ядре Linux для обнаружения утечек памяти. Он работает подобно трассирующему сборщику мусора, сканируя динамически выделенную память в ядре и сообщая о выделениях без активных ссылок, которые невозможно освободить.

Основные возможности kmemleak:

- Отслеживает возможные утечки памяти в ядре Linux.
- Периодически сканирует память для поиска неосвобождённых объектов.
- Выводит отчёты с трассировкой вызовов, где была выделена память.
- Помогает повысить надёжность системы, выявляя "забытые" объекты.

Включение и настройка kmemleak

Включите в конфигурации ядра следующие параметры:

Опция	Описание	Значение
CONFIG_DEBUG_KMEMLEAK	Включение механизма обнаружения утечек памяти	y
CONFIG_DEBUG_INFO	Включение генерации отладочной информации	y
CONFIG_SLAB	Использование SLAB-аллокатора (обычно включён)	y

Как использовать kmemleak (пример).

1. Монтирование `debugfs`. Для доступа к интерфейсу `kmemleak` выполните:

```
sudo mount -t debugfs none /sys/kernel/debug
```

2. Управление `kmemleak` через `sysfs`. Интерфейс `kmemleak` находится в файле

`/sys/kernel/debug/kmemleak`

Доступные операции:

- Запустите сканирование памяти:

```
echo scan > /sys/kernel/debug/kmemleak
```

- Просмотрите найденные утечки:

```
cat /sys/kernel/debug/kmemleak
```

- Очистите отчёты об утечках:

```
echo clear > /sys/kernel/debug/kmemleak
```

3. Загрузим модуль для тестирования `kmemleak`.

Код модуля:

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/slab.h>

static int leak_init(void)
{
    pr_info("Loading leak module\n");
    kmalloc(32, GFP_KERNEL); // deliberately leaked memory
    return 0;
}

static void leak_exit(void)
{
    pr_info("Unloading leak module\n");
}

module_init(leak_init);
module_exit(leak_exit);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Simple module to test kmemleak");
```

Соберем и загрузим модуль

```
make
sudo insmod leak.ko
```

Запустим сканирование памяти:

```
echo scan > /sys/kernel/debug/kmemleak
```

Анализируем отчет

```
sudo cat /sys/kernel/debug/kmemleak
[sudo] пароль для student:
unreferenced object 0xffff888035f095e0 (size 32):
  comm "insmod", pid 872, jiffies 4295002131 (age 239.647s)
  hex dump (first 32 bytes):
    5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a  ZZZZZZZZZZZZZZZZ
    5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a a5  ZZZZZZZZZZZZZZZZ.
  backtrace:
    [<000000000932fd5c9>] kmalloc_trace+0x26/0x90
```

```
[<0000000008c2d70c>] leak_init+0x27/0xfef [test_kmemleak]
[<00000000023fb8ab2>] do_one_initcall+0x56/0x220
[<000000000c5718457>] do_init_module+0x4a/0x1e0
[<0000000004a913bfc>] __do_sys_finit_module+0x9e/0xf0
[<000000000ff1b009d>] do_syscall_64+0x34/0x80
[<0000000008f77b850>] entry_SYSCALL_64_after_hwframe+0x6e/0xd8
```

Что это значит?

```
unreferenced object 0xffff888035f095e0 (size 32):
```

Kmemleak обнаружил объект в памяти размером 32 байта, на который в данный момент нет ссылок (unreferenced). То есть, это подозрительная утечка памяти.

```
comm "insmod", pid 872, jiffies 4295002131 (age 239.647s)
```

Процесс, который выделил эту память — insmod (загрузка модуля ядра), с PID 872. Объект живет уже около 239 секунд.

```
hex dump (first 32 bytes):
```

Приведен шестнадцатеричный дамп первых 32 байт содержимого объекта. В вашем случае, заполнено байтами с кодом 5a (Z в ASCII), что часто используется для инициализации или отладки.

```
backtrace:
```

Трассировка стека вызовов, которая показывает, где именно был выделен данный объект:

- kmalloc_trace — компиляция kmalloc с трассировкой (KASAN/kmemleak).
- leak_init в вашем модуле test_kmemleak.
- Последовательные вызовы ядра для загрузки модуля (do_one_initcall, do_init_module и др.).

Ресурсы для самостоятельного изучения kmemleak.

- [Kernel Memory Leak Detector — kernel.org](https://kernel.org/doc/Documentation/kmemleak/)
- [Finding kernel memory leaks using kmemleak — RDK Central](https://rdkcentral.com/finding-kernel-memory-leaks-using-kmemleak/)
- [Debugging — The Linux Kernel documentation](https://lwn.net/Articles/704000/)
- [Kernel memory leak detector — LWN.net](https://lwn.net/Articles/704000/)

Отладка с использованием Kprobe

Kprobe — это динамический механизм трассировки ядра Linux, который позволяет вставлять пробные точки (пробы) в произвольные места ядра для сбора данных, отладки и анализа. Он позволяет прервать выполнение ядра в заданной точке, выполнить пользовательский обработчик (handler), а затем корректно продолжить работу ядра без перезагрузки.

Основные возможности Kprobe

- Вставка точек прерывания в любое место в ядре по адресу или имени функции.
- Выполнение пользовательских функций до входа в функцию (pre_handler) и после выхода (post_handler).
- Поддержка доступа к аргументам и регистрам прерванной функции.
- Основа для систем динамической трассировки, таких как SystemTap, ftrace и eBPF.
- Позволяет отлаживать ядро и сторонние модули «на лету», без перезагрузки.

Включение и настройка Kprobe

Для использования Kprobes в конфигурации ядра должны быть включены следующие ключевые опции:

Опция	Описание	Значение
CONFIG_KPROBES	Включить поддержку Kprobes	y
CONFIG_KPROBE_EVENTS	Включить поддержку событий Kprobe	y
CONFIG_UPROBE_EVENTS	Включить поддержку пользовательских проб (uprobes)	y
CONFIG_KRETPROBES	Включить поддержку возвратных проб (kretprobes)	y
CONFIG_DYNAMIC_FTRACE	Включить динамическую трассировку (обычно требуется)	y
CONFIG_MODULES	Включить поддержку загрузки и выгрузки модулей (если используются)	y
CONFIG_KALLSYMS	Включить поддержку kallsyms для разрешения символов	y

Виды Kprobe

Вид	Описание
kprobe	Основная проба, вставляется в произвольное место ядра
jprobe	Специальная проба, вставляется в начало функции, удобна для доступа к аргументам (использует setjmp/longjmp)
kretprobe	Проба, срабатывающая при выходе из функции, даёт доступ к значению возвращаемого результата

Как работает Kprobe

- Kprobe устанавливает точку прерывания в заданном месте ядра (адресе инструкции или имени функции), заменяя байт инструкции на breakpoint.
- При достижении этой точки управление передаётся в `pre_handler`.
- После выполнения `pre_handler` оригинальная инструкция выполняется одношагово.
- После выполнения инструкции вызывается `post_handler`.
- Затем управление ядра продолжается как обычно.

Как использовать Kprobe (пример).

Пример простого модуля, который регистрирует Kprobe с pre и post обработчиками:

```
#include <linux/module.h>
#include <linux/kprobes.h>
#include <linux/string.h>

static unsigned int counter = 0;

static int handler_pre(struct kprobe *p, struct pt_regs *regs)
{
    char fname[256];
    char __user *user_fname = (char __user *)regs->si;

    if (strncpy_from_user(fname, user_fname, sizeof(fname)) > 0) {
        fname[sizeof(fname) - 1] = '\0';
        if (strcmp(fname, "new_file.txt") == 0)
            pr_info("Kprobe pre_handler: counter=%u filename=%s\n", counter++, fname);
    }
    return 0;
}

static struct kprobe kp = {
    .pre_handler = handler_pre,
    .symbol_name = "do_sys_openat2"
};

static int __init handler_init(void)
{
    int ret;

    ret = register_kprobe(&kp);
    if (ret < 0)
        pr_info("register_kprobe failed, returned %d\n", ret);
    pr_info("Registered kprobe at %p\n", kp.addr);
    return ret < 0 ? ret : 0;
}

static void __exit handler_exit(void)
```



```
{
    unregister_kprobe(&kp);
    pr_info("Unregistered kprobe\n");
}

module_init(handler_init);
module_exit(handler_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("KGD");
MODULE_DESCRIPTION("Kprobe example filtering 'new_file.txt'");
```

Примечание:

Когда вы регистрируете Kprobe с помощью `register_kprobe()`, вы указываете точку пробы — это может быть адрес инструкции или имя функции ядра.

Kprobe внутренняя реализация заменяет первый байт целевой инструкции (часто в начале функции) на специальную инструкцию прерывания (например, `int3` на x86).

Когда CPU доходит до этой инструкции прерывания, происходит ловля исключения — ядро берёт управление и вызывает `pre_handler` вашего Kprobe.

`pre_handler`

- Вызывается до выполнения инструкции, на которую поставлен Kprobe.
- Позволяет наблюдать состояние регистров, изменять их (аккуратно!), или просто регистрировать события.
- После выполнения `pre_handler` ядро одношагово выполняет заменённую инструкцию (вместо замены breakpoint'a его временно disables для выполнения).
- Это и есть причина, почему Kprobe считается вставленным "перед" инструкцией.

`post_handler`

- Вызывается после одношагового выполнения инструкции, на которую поставлена проба.
- Позволяет собирать информацию о состоянии после выполнения инструкции: значения регистров, результат, обновления и т.д.
- Затем управление возвращается в поток исполнения ядра на следующую за инструкцией.

Допустим мы хотим поставить Kprobe на адрес внутри какой-то функции.

```
static struct kprobe kp = {
    .addr = (kprobe_opcode_t *)0xffffffff8120c3b0, // замените на актуальный адрес инструкции
    .pre_handler = handler_pre,
    .post_handler = handler_post,
};
```

Как узнать адрес инструкции внутри функции?

- Используйте `cat /proc/kallsyms | grep имя_функции`, чтобы получить адрес начала функции.
- С помощью `objdump` или `gdb` с символами ядра получите дизассемблированный код функции и найдите нужный оффсет.
- Адрес для `Kprobe` = адрес начала функции + оффсет инструкции.

Ресурсы для изучения Kprobe

- [Kernel Probes \(Kprobes\) — официальная документация Linux](#)
- [Статья LWN: An introduction to KProbes](#)
- [Обзор Kprobes на habr.com](#)
- [Linux Kernel Labs: Debugging](#)

Отладка и трассировка с помощью Ftrace

Ftrace — это встроенный во всё ядро Linux фреймворк для динамической трассировки функций и событий ядра для целей отладки, профилирования и анализа производительности. Он позволяет разработчикам видеть, какие функции и когда вызываются, сколько времени они занимают, и способствует быстрому выявлению узких мест и ошибок.

Основные возможности Ftrace.

- Трассировка вызовов функций ядра (Function Trace).
- Построение графов вызовов с подсчётом времени на каждый вызов.
- Отслеживание переключений контекста, прерываний и событий ядра.
- Фильтрация трассировки по функциям, процессам и CPU.
- Использование минимальных накладных расходов при выключенном режиме.
- Поддержка динамического включения/отключения трассировки через файловую систему `debugfs`.

Включение и настройка Ftrace

Для функциональной трассировки ядра с помощью ftrace в конфигурации ядра должны быть активированы следующие опции:

Опция	Описание	Значение
CONFIG_FTRACE	Включить поддержку ftrace	y
CONFIG_DYNAMIC_FTRACE	Включить динамическое управление трассировкой	y
CONFIG_FUNCTION_TRACER	Включить трассировку вызова функций	y
CONFIG_FUNCTION_GRAPH_TRACER	Включить граф вызовов функций	y
CONFIG_KALLSYMS	Включить поддержку kallsyms для разрешения символов	y
CONFIG_FTRACE_SYSCALLS	Включить трассировку системных вызовов	y
CONFIG_DYNAMIC_FTRACE_WITH_REGS	Включить поддержку динамической трассировки с регистрами	y

Как работает Ftrace

- При компиляции ядра с поддержкой Ftrace компилятор вставляет вызов функции **fentry()** в начала функций.
- В рантайме Ftrace может динамически изменять поведение этих вызовов (заменяя инструкцию в памяти), чтобы включать или отключать трассировку.
- Данные трассировки записываются в кольцевой буфер, доступный через debugfs.
- Для включения трассировки нужно просто выбрать трассировщик (например, function или function_graph) через специальные файлы в /sys/kernel/debug/tracing/.

Важные файлы и директории

- /sys/kernel/debug/tracing/ — главный интерфейс.
- current_tracer — файл для выбора активного трассировщика.
- trace — содержит текущий вывод трассировки.
- set_ftrace_filter — фильтр функций по именам.
- tracing_on — включает и выключает запись трассировки.
- trace_pipe — "живой" вывод трассировки для чтения в реальном времени.

Как использовать Ftrace (пример).

Можно использовать следующий bash-скрипт

```
#!/bin/sh
```

```

dir=/sys/kernel/debug/tracing

if [ -z "$1" ]; then
    echo "Usage: $0 function_name"
    exit 1
fi

NAME_FUNC=$1

sysctl kernel.ftrace_enabled=1
echo function_graph > ${dir}/current_tracer
echo "$NAME_FUNC" > ${dir}/set_ftrace_filter
echo 1 > ${dir}/tracing_on
sleep 1
echo 0 > ${dir}/tracing_on
less ${dir}/trace
echo > ${dir}/set_ftrace_filter
echo > ${dir}/trace

```

Примечание:

Ftrace поддерживает несколько видов трейсировщиков, каждый из которых решает разные задачи. Некоторые из основных:

Трейсер	Описание
nop	«Пустой» трейсировщик, который не делает ничего. Используется для выключения трассировки без выгрузки настроек.
function	Трассировка вызовов функций ядра — выводит имя каждой вызванной функции без аргументов. Самый простой и распространённый.
function_graph	Продвинутая трассировка функций — строит граф вызовов и отображает информацию о времени выполнения каждой функции и подвызовах.
blk	Трассировка событий блочного ввода-вывода, показывает вызовы и латентность операций с дисками.
wakeup	Трассировка пробуждений задач (wakeup events), полезна для анализа планирования.
wakeup_dl	Анализ задержек пробуждения с дополнительной информацией по deadline.
wakeup_rt	Анализ пробуждений реального времени (для real-time систем).
irqsoff	Трассировка длительных отключений прерываний (IRQ off). Помогает анализировать блокировки.
preemptoff	Отслеживание времени отключения вытеснения задач (preemption off).
power	Трассировка событий, связанных с управлением энергопотреблением и переходом в спящие состояния.
sched	Трассировка событий планировщика — переключения задач, задержки и др.

Эти трейсировщики можно переключать в файле `/sys/kernel/debug/tracing/current_tracer`.

Для просмотра списка доступных трейсировщиков можно прочитать файл

`/sys/kernel/debug/tracing/available_tracers`.

Ресурсы для изучения Ftrace.

- [Трассировка ядра с ftrace — статья на Хабре](#)
Подробное описание возможностей и примеры работы с ftrace.
- [Трассировка и профилирование ядра Linux — Selectel PDF](#)
Краткое руководство с примерами по использованию ftrace.
- [Трассировщик функций ftrace — Protokols.ru](#)
Обзор основных функций, настройка tracefs и примеры.
- [Официальная документация Linux Kernel по ftrace](#)
Полное техническое описание, параметры конфигурации и примеры.
- [Linux Kernel Labs: Debugging Tools](#)
Учебные материалы по отладке и трассировке ядра Linux.
- [Использование ftrace в Android Kernel](#)
Руководство по применению ftrace в ядре Android.
- [Документация Red Hat Enterprise Linux о ftrace](#)
Руководство для разработчиков с примерами и best practices.

Отладка с использованием bpftrace

bpftrace — это высокоуровневый язык трассировки и инструмент для динамического анализа и отладки Linux, построенный поверх eBPF. Он позволяет прямо из командной строки создавать и запускать мощные программы трассировки с очень простой и лаконичной синтаксической структурой, похожей на AWK.

Основные возможности bpftrace

- Динамическая трассировка вызовов функций, системных вызовов, точек трассировки (tracepoints), kprobes, uprobes.
- Обеспечивает возможность легко собирать статистику, подсчитывать частоты, измерять временные интервалы.
- Позволяет просматривать данные в реальном времени с помощью удобных агрегатов: гистограмм, хэшей, счетчиков.
- Поддерживает трассировку пространств ядра и пользовательских процессов.
- Использует JIT-компиляцию для высокой производительности.
- Позволяет создавать программы очень коротко и просто — от одной строки в командной строке.
- Основа для быстрого прототипирования и ad-hoc отладки.

Включение и настройка BPF

Для успешного использования eBPF и bpftrace в конфигурации ядра Linux должны быть включены следующие ключевые опции:

Опция	Описание	Значение
CONFIG_BPF	Включить базовую поддержку BPF	y
CONFIG_BPF_SYSCALL	Включить системный вызов для BPF	y
CONFIG_BPF_JIT	Включить JIT-компиляцию BPF-программ	y
CONFIG_BPF_EVENTS	Включить поддержку BPF-событий	y
CONFIG_BPF_STREAM_PARSER	Включить поддержку стримингового парсера BPF	y
CONFIG_BPF_DISPATCHER	Включить поддержку диспетчера BPF	y
CONFIG_NETFILTER_XT_MATCH_BPF	Включить BPF в netfilter (для сетевого фильтра)	y
CONFIG_BPF_LSM	Включить поддержку BPF для LSM	y
CONFIG_DEBUG_FS	Включить debugfs (необходим для функций трассировки)	y
CONFIG_TRACEPOINTS	Включить поддержку точек трассировки	y

Включение и настройка BPF maps

Для работы с BPF maps в ядре Linux должны быть включены следующие опции:

Опция	Описание	Значение
CONFIG_BPF_MAPS	Включить поддержку BPF maps	y
CONFIG_BPF_HASH	Включить поддержку хеш-таблиц BPF	y
CONFIG_BPF_ARRAY	Включить поддержку массивов BPF	y
CONFIG_BPF_LRU_CACHE	Включить поддержку LRU-кэша для BPF карт	y
CONFIG_BPF_LRU_PERCPU_CACHE	Включить поддержку LRU-кэша с учетом CPU	y
CONFIG_BPF_STACK_MAP	Включить поддержку стэков BPF maps	y
CONFIG_BPF_JIT	Включить JIT-компиляцию BPF программ для повышения производительности	y

BPF maps — это специальные структуры данных, используемые в BPF (Berkeley Packet Filter) и его развитии eBPF, которые обеспечивают разделяемую память для хранения данных. Они представляют собой объекты ядра, реализующие различные виды хранилищ, как правило, в формате ключ-значение, и служат для обмена данными между BPF-программами, работающими в пространстве ядра, и пользовательским пространством. BPF maps позволяют программам сохранять состояние между вызовами, а также делиться этими данными с другими программами и приложениями. Доступ к этим мапам осуществляется через системный вызов bpf(2) или специальные функции-помощники ядра.

Существует множество типов BPF maps, включая хэш-таблицы, массивы, деревья поиска и другие специализированные виды, которые расширяют возможности взаимодействия программ и ядра. Эта система играет ключевую роль в архитектуре eBPF, позволяя программам эффективно и безопасно хранить и обмениваться данными, например, для сетевого мониторинга, трассировки и других задач в рамках ядра Linux.

Как использовать bpftrace (пример).

Посмотрим и разберем код `trace_exec.bt`:

```
#!/usr/bin/bpftrace

BEGIN {
    printf("Tracing execve syscalls and do_sys_openat2 kprobe... Hit Ctrl-C to exit.\n");
}

tracepoint:syscalls:sys_enter_execve
{
    printf("PID %d (%s) вызвал execve: %s\n", pid, comm, str(args->filename));
}

kprobe:do_sys_openat2
{
    @[comm] = count();
}

interval:s:5
{
    print(@);
    clear(@);
}
```

Трассировка события `execve`

- Строка `tracepoint:syscalls:sys_enter_execve` задаёт точку наблюдения на системный вызов `execve`, когда процесс начинает выполнение новой программы.
- Каждый раз, когда такой системный вызов начинается, вызывается блок внутри `{ ... }`.
- В нём выполняется `printf`, который выводит `PID` процесса (`pid`), имя процесса (`comm`) и имя исполняемого файла (`args->filename`).

Функция `str()` преобразует пользовательскую строку к обычной строке для вывода.

Подсчёт количества вызовов функции `do_sys_openat2`

`kprobe:do_sys_openat2` устанавливает пробу (`kprobe`) на начало функции ядра `do_sys_open`, которая отвечает за открытие файлов.

Каждый вызов `do_sys_open` инкрементирует счётчик для текущего имени процесса через агрегат `@[comm] = count();`. Эта ассоциативная переменная хранит счётчики вызовов для каждого процесса отдельно.

Периодический вывод статистики

`interval:s:5` задаёт действие, выполняющееся каждую секунду (через 5 секунд).

В блоке вызывается `print(@);`, который выводит текущие значения счетчиков вызовов `do_sys_openat2` для всех наблюдаемых процессов.

`clear(@);` сбрасывает счётчики, чтобы начать новый период подсчёта.

Что происходит в итоге?

- Скрипт в реальном времени отслеживает все вызовы запуска программ через `execve`, выводя информацию о каждом запуске.
- Параллельно ведётся подсчёт, сколько раз каждый процесс вызывает открытие файлов.
- Каждые 5 секунд выводится статистика по открытию файлов по процессам, после чего счётчики обнуляются для сбора новой статистики.
- Используется комбинирование `tracpoint` и `kprobe` для детальной трассировки разных событий ядра.

Отличия в типах проб (probes)

- **tracpoint**: срабатывает на заранее определённые события ядра, например, системные вызовы.
- **kprobe**: динамическая проба на функцию ядра, в данном случае `do_sys_open`.

Ресурсы для изучения bpftrace

- [Повышайте уровень с bpftrace: удобная трассировка eBPF в 2025](#)
Статья с обзором возможностей, примерами и нестандартными способами использования.
- [Отлаживаем ядро из командной строки с bpftrace — Хабр](#)
Пример использования, синтаксис и возможности bpftrace.
- [Основы трассировки с помощью bpftrace — Protokols.ru](#)
Руководство по основам и настройке.
- [Официальный репозиторий bpftrace на GitHub](#)
Исходный код, документация и примеры.
- [Руководство пасечника или обзор инструментария eBPF](#)
Обширный обзор инструментов eBPF, включая bpftrace.
- [Отлаживаем ядро из командной строки с bpftrace — Temofeev.ru](#)
Рекомендации по использованию в реальных сценариях.
- [Основы и примеры bpftrace на русском от DesdeLinux](#)
Хорошее введение с практическими примерами.

Итоговая таблица

Инструмент	Цель использования	Основные возможности	Когда использовать
KGDB	Пошаговая отладка ядра с GDB	Полноценная отладка ядра, установка breakpoint	Когда нужна глубокая пошаговая отладка ядра, особенно в ранней загрузке и модулях
KASAN	Обнаружение ошибок обращения с памятью	Автоматический поиск ошибок использования памяти	При поиске багов с памятью: выход за границы, double free и др.
kmemleak	Поиск утечек памяти в ядре	Автоматический поиск «забытых» областей памяти	При подозрении на утечки памяти, долгоживущие объекты без ссылки
Kprobe	Вставка динамических проб (пробные точки) в ядро	Отслеживание вызовов функций, доступ к регистрам	Когда нужно отслеживать или модифицировать выполнение отдельных функций на лету
Ftrace	Общая динамическая трассировка ядра	Отслеживание вызовов, граф вызовов, события	Для комплексного анализа производительности, отслеживания системных вызовов и событий
bpfftrace	Высокоуровневая динамическая трассировка на базе eBPF	Простое описание трассировок, подсчёты, гистограммы	При необходимости быстрой отладки и анализа с минимальным написанием кода